

Live-coding feasibility for purerl-tidal

A design exploration of whether and how to evolve purerl-tidal toward GHCi-flavored live coding through a PureScript-host eDSL. Captured 2026-05-05 from a conversation with Claude. Meant as a reference for re-engaging the question later, not as an action plan.

This doc is deliberately verbose. It assumes the reader is fluent in PureScript and FP idioms but **not** an Erlang/BEAM expert and not deeply familiar with TidalCycles internals. Background sections (Appendices A and B) explain those from scratch.

The conclusion the conversation reached: **don't fork, don't redesign. Keep evolving the current parser-DSL architecture, refactor the BEAM side toward per-track gen_servers, and revisit the eDSL question when either (a) we hit a capability limit, or (b) compile-latency drops to the point where it's free to take.**

1. What live coding actually demands

The defining property is **sub-second feedback from edit to sound**. A live coder types an expression, hits a keystroke, and hears the result either immediately or on the next bar boundary. Latency breaks into:

- **Author latency** — keystroke → "this is the new pattern." Includes parsing, type-checking, and producing a runtime value. TidalCycles in GHCi is typically 100–300 ms.
- **Schedule latency** — "new pattern" → events queued for output. Microseconds in any decent design.
- **Audio latency** — event-queued → sound-out-of-speakers. Sub-millisecond. Handled by an audio runtime with kernel timestamps (SuperCollider, link-spike, etc.).

The hard constraint is on **author latency**. Schedule and audio latency are solved problems. As a rough scale of usability:

Author latency	Live-coding feel
~200 ms (GHCi reference)	Free improvisation, real-time pattern reshaping
500 ms – 1 s	Comfortable live coding, occasional friction
1 – 2 s	Composition-flavored; each fire is a deliberate moment
2 – 3 s	Pre-rehearsed sets with periodic tweaks
5 – 10 s	Compose-then-fire as an explicit cadence; not Tidal-style

If author latency exceeds ~500 ms, **free improvisation breaks**: you can no longer think-and-type at musical speed. If it exceeds ~2 s, you can still compose-then-fire fluidly but lose the responsive-to-the-moment shape that defines Tidal's culture.

2. The current purerl-tidal stack

Four pieces, four runtimes, OSC and WebSocket between them.

purperl-tidal (BEAM/Erlang VM, port :3012). PureScript code compiled via `purs-backend-erl` to Erlang `.beam` files. Provides:

- A Cowboy WebSocket server (`src/Tidal/WebSocket/Handler.erl`) for receiving live-coding messages from Calypso and other clients.
- A single `MIDIScheduler` Erlang process (in `Tidal.MIDIScheduler`) that holds all current "tracks" (running patterns) and emits events on a tight loop with look-ahead-ms scheduling.
- Pattern math (`Tidal.Pattern.Core`, `.Branched`, `.Types`) ported from upstream Haskell `TidalCycles`.
- A mini-notation parser (`Tidal.Parse.Parser`) also ported from upstream.
- A small host-language layer (`Tidal.Expr`) for `:` -prefixed expressions like `:every 4 rev "bd sn"`.

Calypso (Halogen frontend, port :3061; HTTP server :3060). The user-facing editor — composition pane, cell pane, fire UI, ghost-text proposals, etc. Sends source text to `purperl-tidal` over WebSocket.

link-spike (Rust, separate process). Owns the audio-side hard real-time work — receives OSC, dispatches CoreMIDI with kernel-timestamped notes, manages Ableton Link tempo synchronization.

cv-router (Rust, separate process). Drives ES-9 CV/Gate output to the modular synth via CoreAudio.

The split between BEAM (pattern math, scheduling decisions) and Rust (kernel-timestamped audio dispatch) is **already the GHCi/SuperDirt split done well**. We didn't realise at the time that's what we were building, but the architecture is correct: BEAM can pause for GC without affecting the audio path because the audio path doesn't run in BEAM.

3. The three grammars in current `purperl-tidal`

When a live coder types a line into Calypso, three different parsers may engage depending on the line's shape. Knowing this layering is prerequisite for understanding any future change.

3.1 Verb layer

Top-level commands the rig understands as "do this thing." Examples:

- `gate <ch> <pattern>` — fire pattern as drum-kit gates on channel
- `cv <bus> <pattern>` — fire as control voltage on bus
- `bind <name> <action-spec>` — register a named binding
- `bpm <n>` — set tempo
- `state` — read current scheduler snapshot
- `hush` — silence everything

This is the **wire protocol** of the rig. Implemented in `src/Tidal/WebSocket/Handler.erl`'s `try_parse_prefixed` function. Hand-rolled Erlang. Each verb is a discrete dispatch — no shared parsing infrastructure.

3.2 Host language (the `:` prefix)

After certain verbs (`gate <ch>`, `<bound-name>`), if the body starts with `:` , the rest is parsed as a Haskell-style prefix-application expression by `Tidal.Expr.parseExpr`. Examples:

```
gate 10 :every 4 rev "bd sn hh cp"
bass :mult [L:id, R:rev] "c4 e4 g4 b4"
```

The expression evaluator knows a small set of registered names (`id`, `rev`, `palindrome`, `slow`, `fast`, `every`, `jux`, `mult`, `alternate`, `crossfade`, `gate`) and produces a **Pattern String** that the scheduler dispatches.

This is the **compositional surface** for Branched combinators and pattern transforms. Added Step 1–2 of the Branched work (April–May 2026). Parens were added 2026-05-04.

3.3 Mini-notation (inside string literals)

Inside "...", a separate parser handles Tidal's mini-notation: `"bd sn"`, `"a*4"`, `"[a,b,c]"`, `"<a b c>"`, `"c4'major7"`, etc. Direct port of upstream Tidal's mini-notation grammar; the only piece we didn't write from scratch.

The three parsers nest: verb-layer at the line level, host-language inside `:`-prefixed bodies, mini-notation inside string literals.

4. The eDSL question

The conversation that prompted this doc started with "could we replace the parser stack with a single eDSL written in PureScript?" The motivation: layered grammars work but have costs.

- Each layer requires its own parser, error messages, error recovery.
- Boundaries are sources of confusion for users (e.g., colon transforms vs pipe transforms vs prefix application — three syntaxes for "transform this pattern" depending on context).
- Type-checking spans only one layer at a time.
- Refactoring tools (rename, find-usages) work poorly across grammar boundaries.
- New combinators require touching the parser, not just the library.

A full eDSL — what TidalCycles is — replaces all this with: **the user writes in the host language directly, importing pattern combinators as ordinary functions**. TidalCycles users don't write to a custom DSL; they write Haskell. Lambdas, typeclasses, partial application, type inference, LSP completions, the works.

Crucially: the existing `Tidal.Pattern.Core` and `Tidal.Pattern.Branched` modules in `purel-tidal` **already are an eDSL** — they're PureScript imports with the same combinators TidalCycles' Haskell library exposes. What's missing isn't language design; it's the live-coding *pipeline* (compile + evaluate cells fast enough to feel live).

4.1 Tagless final encoding

If we go eDSL, the encoding to use is **finally tagless** — operations expressed as typeclass methods rather than data constructors:

```
class Pattern p where
  silence :: ∀ a. p a
  fast    :: ∀ a. Rational -> p a -> p a
```

```

rev      :: ∀ a. p a -> p a
every    :: ∀ a. Int -> (p a -> p a) -> p a -> p a
-- ...

class Pattern p <= Branched p where
  fanOut :: ∀ a. Array (Tuple Voice (p a -> p a)) -> p a -> Branch p a
  merge  :: ∀ a. Branch p a -> p a

```

Users write polymorphic expressions:

```

mySong :: ∀ p. Branched p => p String
mySong = mult [bass :=> id, lead :=> rev] (str "c4 e4 g4 b4")

```

We provide multiple **interpreters** as instances of **Pattern** / **Branched**:

- An **Events** instance produces an actual **Pattern** value (in-process eval / preview).
- An **AST** instance builds a serializable tree (to ship over the WebSocket to BEAM, decode, dispatch).
- Possibly a **PrettyPrint** instance for debugging.

The advantage over a deeply-embedded ADT: the user writes ordinary PureScript with all the host-language facilities (helper bindings, **where**-clauses, partial application, type inference, ...), and we extract a serializable structure at the end via the **AST** interpreter.

4.2 What you give up

Tagless final has one real cost: **host-language closures that capture runtime values into pattern values can't always be reflected**.

every n (\p -> rev (fast 2 p)) reflects fine — the lambda is $\forall p. \text{Pattern } p \Rightarrow p a \rightarrow p a$, fully polymorphic, applied at AST construction time.

every n (\p -> if x > threshold then rev p else fast 2 p) where $x :: \text{Int}$ is a runtime non-pattern value mostly works (the **if** resolves at host time before reflection — the AST sees only the chosen branch).

But conditioning on a *pattern of booleans* — e.g. **condIf :: p Boolean -> p a -> p a -> p a** — needs to be a typeclass method in the eDSL, not a host-language **if**. You can't lift a Haskell-side **if** into the pattern world.

In TidalCycles practice this is almost never a real limitation. Most pattern lambdas are operator composition; pattern-time conditionals are rare and the existing combinators handle them.

5. The latency floor (measured 2026-05-05)

Question: **how fast can a one-line cell change get from edit to running on BEAM?**

Measured by touching `src/Tidal/Pattern/BranchedExamples.purs` (a representative leaf module that imports **Pattern.Core** + **Pattern.Branched**). All times wall-clock on a 16 GB M-series MBP with warm caches.

5.1 Stage breakdown

The pipeline is: PureScript source → CoreFn JSON → Erlang `.erl` → BEAM `.beam`.

Stage	Tool	Time
spago dependency listing	<code>spago sources</code>	0.3 s
PureScript type-check + CoreFn emit	<code>purs compile --codegen corefn</code>	0.5 s
CoreFn → Erlang	<code>purs-backend-erl</code> (no flags)	5.3 s
Erlang → BEAM (one file)	<code>erlc</code>	0.2 s
Total naked pipeline		~6.3 s

The dominant cost is `purs-backend-erl`'s whole-program optimizer pass. It runs cross-module inlining and dead-code elimination across all 363 modules in the dependency closure (Prelude, the registry's standard library, our pattern modules, the scheduler) on every invocation. It's part of why `purs-backend-erl`'s runtime-output is fast — but it pays for that at compile time.

For comparison: `spago build` does the same plus its own dependency-resolution overhead, totalling 5.8 s.

`make` (the existing Makefile target) was measured at 92 s but that's a Makefile defect — it `erlc`s every `.erl` from scratch every time, no per-file timestamp checks. Not a fundamental cost; fixable with a smarter Makefile or just using rebar3.

5.2 The `--filter` discovery

`purs-backend-erl` accepts `--filter <module-prefix>` to scope the optimizer to just the import closure of the named module. With `--filter Tidal.Pattern.BranchedExamples`:

- Module count: 105 (just the closure of imports the cell needs) instead of 363.
- Time after change: **1.4 s** (down from 5.3 s).
- Time on no-change: 0.24 s.

Filtered pipeline total: ~2.4 s per change.

With further engineering — caching `spago sources` once at boot, narrowing source globs for `purs compile`, possibly running a per-cell daemon that holds the optimizer in-process — realistic best case is **~1.5 s per fire**.

5.3 What this means for path-choice

Loop time	Live-coding feel
200 ms	GHCI baseline; free improvisation
500 ms–1 s	Comfortable live coding
1.5–2 s	Our realistic best case with current toolchain
2–3 s	Achievable with no engineering effort beyond <code>--filter</code>

Loop time	Live-coding feel
6 s	What we'd hit naively

We're at the bottom of "tolerable for many live-coding workflows" with engineering attention. Not GHCI-level, but substantially better than where the conversation started.

Crucially: **the eDSL path isn't structurally blocked**. It's slower than ideal, but it works. That changes the tactical calculus.

6. BEAM features we underutilize

Erlang's BEAM virtual machine has a set of properties that are unusually well-suited to live music — yet purerl-tidal currently uses very few of them. This section walks through them assuming the reader hasn't worked with BEAM commercially.

6.1 Lightweight processes

An Erlang "process" is not an OS thread. It's a VM-managed unit, ~300 bytes each, scheduled cooperatively by the BEAM scheduler across however many OS threads the VM has. A single BEAM node can run millions of processes. ~16 tracks is rounding error.

Each process has its own private heap. Processes communicate only by sending messages — they cannot share memory directly. This is the actor model done well, and it's a million miles from the threading model in JVM/Node/Python.

A process is created with `spawn(fun() -> do_something() end)` and runs concurrently with everything else. It runs until its function returns or it crashes.

6.2 Per-process garbage collection

Because each process has its own heap, **GC pauses one process at a time, not the whole VM**. If `lead`'s pattern allocates heavily and triggers GC on `lead`'s process, only `lead` pauses; `bass` and `pad` continue running undisturbed.

This is THE architectural feature that makes BEAM suitable for soft real-time work despite being garbage-collected. You can have GC, you can have hundreds of tracks, and the audio path stays smooth because no one process holds the world.

By contrast: Node has one heap, one GC, stop-the-world pauses on major GC (5–50 ms typical). Java is similar. GHC's RTS GC is also stop-the-world by default (TidalCycles works because per-cycle allocation is small, not because GHC's GC is friendly).

For live coding: **the per-track decomposition isn't an optimisation, it's the GC isolation story**. Without it, a heavy pattern recompute on one track would briefly pause all the others.

6.3 Hot code loading

BEAM supports **swapping a module's code while processes are running**. The system call `code:load_file(Module)` reads the latest `.beam` file and installs it. Existing call frames in running processes finish on the old code; subsequent calls resolve to the new code.

This is **the BEAM equivalent of GHCi's "evaluate this in the running session."** A new pattern combinator gets compiled to a `.beam` file, hot-loaded, and the next pattern construction picks it up. Running tracks finish their current cycle on old code, next cycle on new. Sub-frame transition.

We do not currently use this. Today's purerl-tidal restart-the-VM model is the equivalent of restarting GHCi on every code change — possible but slow and stateful (loses bindings, slot values, etc.).

6.4 OTP supervision

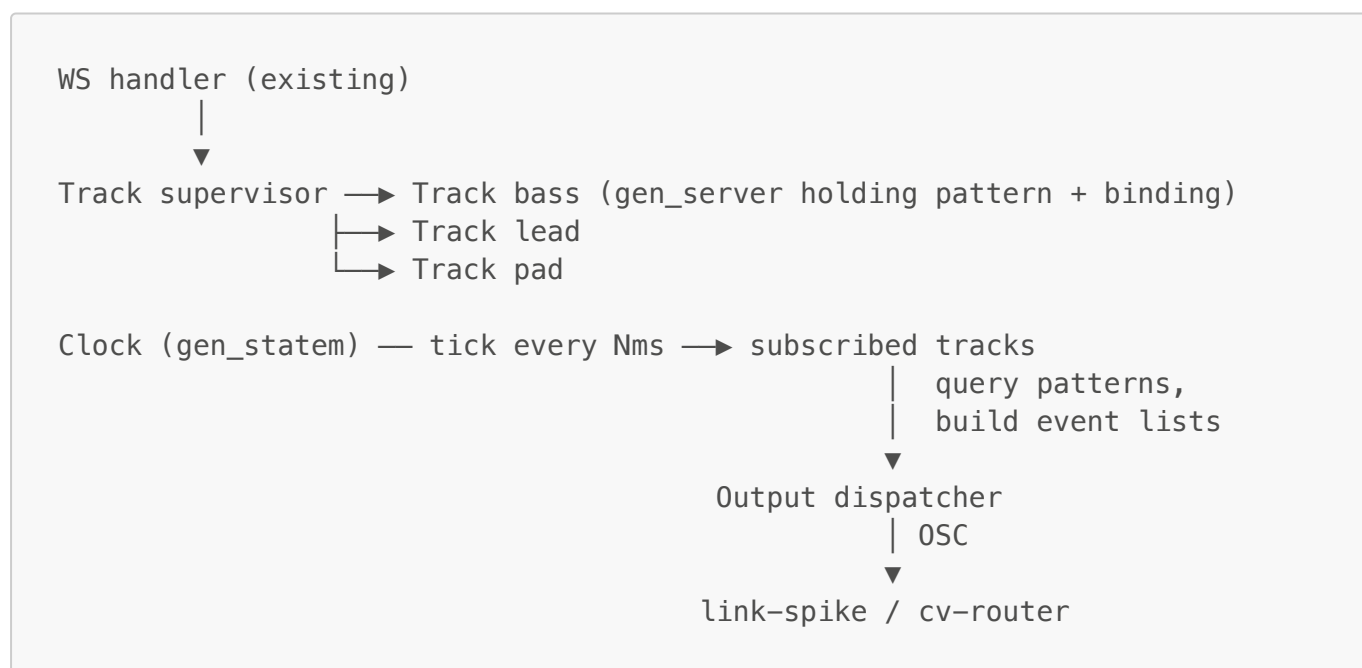
The OTP framework (Open Telecom Platform — shipped with Erlang) provides standard process patterns:

- **gen_server** — request/response server with state. Clients call `gen_server:call(Pid, Request)` (synchronous, returns the reply) or `gen_server:cast(Pid, Notification)` (fire-and-forget).
- **gen_statem** — state machine.
- **supervisor** — restart-on-crash policy for child processes. When a worker crashes, the supervisor restarts it according to a strategy: `one_for_one` (just this one), `one_for_all` (all siblings), `rest_for_one` (this and later-started siblings).

A typical BEAM application is a tree of supervisors and workers. Crashes are local; the system recovers automatically.

For live coding: **a buggy pattern crashes its track, supervisor restarts it, other tracks keep playing.** Today's monolithic scheduler has no isolation — a crash anywhere takes everything down. Live coders write broken patterns *all the time*; the current architecture is fragile in a way that BEAM specifically solves.

6.5 The decomposition we should pursue



Each Track is a `gen_server` holding its current Pattern + Binding. On a clock tick it queries its pattern for the next sub-window of events and forwards them to the output dispatcher. Live re-fire is `gen_server:cast(Track, {set_pattern, P})` — one message to one process, atomic, surgical.

What this buys, concretely:

- **Crash isolation** — buggy `lead` doesn't kill `bass`.
- **GC isolation** — heavy allocation on one track doesn't pause others.
- **Live module reload via `code:load_file`** — no VM restart for new combinators. New combinator? Compile, hot-load, next cycle picks it up.
- **Distributed-Erlang as a free property** — tracks can move between nodes if you ever wanted (probably overkill for one laptop, but the option costs nothing). Local and distributed look identical when the message-passing model is right; this is what 90s distributed-systems work taught everyone.

6.6 What this asks of Purerl

The refactor pattern is "Erlang shell, PureScript filling." Each `gen_server` is a thin Erlang module providing the OTP callbacks (`init/1`, `handle_call/3`, `handle_cast/2`, ~30 lines of boilerplate) that calls into PureScript modules for the actual pattern math.

This is already the pattern we use for FFI files (`tidal_log.erl`, `tidal_link_anchor.erl`, `tidal_stateBus@foreign.erl`, `tidal_mIDIBridge@foreign.erl`). The decomposition is just bigger application of the same shape. Translation cost is real but bounded — probably 1–2 weeks for the full refactor.

7. Three paths forward

Synthesising everything: there are three coherent directions.

7.1 Path A — Polish the current architecture

Status: default. No structural changes; refine what's there.

What it entails:

1. Move the scheduler to per-track `gen_servers` + supervisor (§6.5).
2. Hot-code-load support for new combinators.
3. Document the verb / host / mini-notation grammar boundaries clearly.
4. Continue extending the host language (more combinators registered in `Tidal.Expr`, the multi-destination Branched work currently in flight).
5. Optionally: a `--filter`-based daemon for cell rebuilds, getting fire latency to ~1.5–2 s.

What you get: a robust, crash-isolated, GC-isolated live-coding rig with a parser-based DSL. Capability-comparable to TidalCycles though not elegance-comparable — you can do everything TidalCycles can, just with more parser overhead.

What you don't get: PureScript host-language expressivity inside cells. No closures, no typeclasses, no LSP support across cell boundaries.

Effort: moderate. The OTP refactor is real work but bounded.

7.2 Path B — Tagless-final eDSL via purerl, BEAM stays

Status: plausible. The numbers say tractable.

What it entails:

1. Everything from Path A.
2. Cells become PureScript modules in a per-cell sub-project (or inline with the workspace).
3. Cells use tagless-final encoding (Pattern as typeclass).
4. The fire action: edit cell → `purs compile --codegen corefn` → `purs-backend-erl --filter <Cell>` → `erlc` → `code:load_file` → send pattern message to track gen_server.
5. Per-cell-fire latency: ~1.5–2.4 s with engineering.
6. AST interpreter as a typeclass instance for serializing across the WS boundary.

What you get: real PureScript inside cells. Lambdas, typeclasses, helper functions, type inference, LSP, refactoring tools. The whole host-language stack. Plus all the BEAM benefits from Path A.

What you don't get: GHCi-comparable latency. ~2 s is workable but not the responsiveness Tidal users expect. Free improvisation remains out of reach.

Effort: large. Path A's refactor + a build-pipeline daemon + cell-as-module convention + tagless encoding of all combinators + AST interpreter + WS-shipping serialization.

7.3 Path C — PureScript-on-Node, decoupled dispatch

Status: speculative. Untested.

What it entails:

1. Cells run in Node (V8 JS runtime), compiled via PureScript JS backend.
2. Pattern math evaluates in JS; emits events with future timestamps.
3. Events stream to link-spike (already kernel-timestamping) or directly to BEAM for dispatch.
4. BEAM holds long-running state (bindings, slots, BPM) but doesn't run cell code.

What you get: potentially fastest cell-fire latency (untested but the JS backend has more compiler attention than `purs-backend-erl`). Full PureScript including real closures captured into patterns. Decouples live-coding from BEAM entirely.

What you don't get: a single coherent runtime story. Two different VMs share the rig. Today blocked by CommonJS-deprecation errors in PureScript dependency packages — the JS backend won't compile against current Prelude until upstream packages migrate to ESM. Could be fixed but it's external work.

Effort: very large. New runtime, new dispatch path, dependency-package migration as prerequisite.

8. Recommendation

Path A first, eDSL question parked.

Reasoning:

- We haven't hit a capability limit driving us off the current architecture. The "I want eDSL" pull is ideological, not forced. Acknowledged as such by Andrew in the conversation that prompted this doc.
- Path A gives us crash-isolation and live-update on the existing parser-DSL. That's a genuine quality improvement independent of the eDSL question.

- Once Path A lands, Path B becomes a smaller delta — the per-track infrastructure and hot-load story are reusable.
- The fire-latency measurement is tractable enough that Path B isn't blocked even if we don't pursue it now. We can always revisit.

Concrete plan:

1. Continue extending the parser-DSL surface with what users want next (multi-destination dispatch, more combinators, the tour examples actually sounding melodic — work in flight).
2. Refactor scheduler to per-track `gen_servers` + supervisor + clock + dispatcher when there's appetite.
3. Add `code:load_file`-based hot reload for the combinator modules.
4. Park the eDSL pursuit as a documented option (this doc), revisit when (a) the parser-DSL hits a capability limit, or (b) we want the LSP/typeclass/refactor tooling for cells strongly enough to commit the engineering.

9. Open questions

Surfaced but not settled in the conversation:

- **Phase-locked re-fire:** when a cell recompiles mid-cycle, does the track's playhead carry over or reset? TidalCycles' answer is "carry over." Worth choosing the same and being explicit.
- **Cell module shape:** a top-level binding `myCell :: Pattern String`? A whole module with helpers + one export? Both work; the first feels most live-coding-ish.
- **Mute as a separate axis:** a *time-axis* concept (this voice is silent for now), distinct from the source-axis decoupling discussed elsewhere. Worth its own design pass.
- **Decoupling source-of-fire from cell creation:** the "make-cell-and-fire" friction in the current Calypso UI suggests "fire any line of source" should exist independently of cell creation. Cells become editor scratchpads; the unit of fire is "a line of composition source." This is a parallel design thread, valuable independent of the eDSL question.
- **Multi-destination Branched dispatch:** voice-tags as binding-names rather than internal labels (`mult [bass:id, lead:rev] "c4 e4 g4 b4"` — bass and lead each get their own `BoundTrack`, no outer prefix). This is the multi-destination work currently in flight and was paused to write this doc.
- **Collaborative live-coding:** BEAM's distributed-Erlang property means multiple users on different machines could edit and fire on the same rig with native message-passing. Not currently planned but mentioned as interesting.

Appendix A: TidalCycles internals for reference

For readers without prior exposure to how TidalCycles actually works under the hood.

A.1 What TidalCycles is

A Haskell library for live-coding music, written by Alex McLean starting in 2009. The user installs it as a Haskell package and runs it through GHCi (Haskell's REPL).

A.2 What live-coding TidalCycles looks like in practice

The user opens a text editor with a TidalCycles plugin (Atom/VSCode/Vim/Emacs). The plugin starts a GHCi session in the background and connects the editor's current buffer to it.

The user types something like:

```
d1 $ fast 4 $ rev "bd sn cp hh"
```

and hits Ctrl-Enter on that line. The plugin sends the line as text to GHCi's stdin. GHCi:

1. Lexes and parses the text as a Haskell expression.
2. Type-checks against the loaded environment (which includes `Sound.Tidal.Context` from the TidalCycles library).
3. Evaluates the expression. The expression's effect is to mutate a global "Stream" object that holds the currently-running patterns per channel (`d1` is the channel — d-one), spawning a thread that queries the pattern for the next bar of events and sends them via OSC.
4. Returns control to the prompt, ready for the next expression.

The whole loop is typically 100–300 ms.

A.3 What SuperDirt is

A SuperCollider program — a long-running process running in parallel — that listens for OSC messages of the shape "play sample X at time T, with parameters P." It's the audio engine. SuperCollider is itself an audio programming language with a very-low-latency real-time scheduler; SuperDirt adds the sample-management layer (a directory of sample files indexed by name, polyphony management, basic synthesis).

A.4 How GHCi's GC doesn't cause audio glitches

Because **GHCi doesn't render audio**. GHCi's job is to compute event lists with future timestamps. SuperDirt renders the audio in SuperCollider's RT thread, which is a different runtime entirely (a different process, often on a different OS-scheduling tier).

GHCi can pause for GC and SuperDirt keeps playing whatever it's already been told to play. The audio path's only requirement is that SuperDirt has events queued at least one audio buffer ahead of the current time, which TidalCycles' look-ahead scheduling ensures.

This is the **decoupled-dispatch architecture** that any live-coding system must have. We accidentally built the same thing in our rig: BEAM does the pattern math, link-spike does the kernel-timestamped MIDI dispatch, the two communicate over OSC, and BEAM's GC pauses don't reach the audio path.

A.5 Why ~200 ms works for GHCi

Because GHC's incremental compilation in-process is well-optimised and most user changes are small (one-line edits to one expression). GHCi has `:reload` semantics that recompile only what changed. Not a magical mechanism — just careful engineering of the compile pipeline within an in-process Haskell environment.

Our analog (purs-backend-erl with `--filter`) is in the same ballpark architecturally but slower because the optimizer pass does cross-module work. The structural reason: GHC's optimizer is per-module, with cross-module inlining gated by `INLINABLE` pragmas; purs-backend-erl's optimizer is whole-program by default.

Appendix B: Erlang quick-reference for non-experts

For readers who haven't worked with Erlang commercially.

B.1 Processes

A "process" in BEAM is an actor — an isolated VM-managed unit with its own heap and message mailbox. To run a function in a new process:

```
Pid = spawn(fun() -> do_something() end).
```

This returns a `Pid` (process ID). The process runs concurrently with everything else. They are **not OS threads** — millions can exist on a single machine. The BEAM scheduler runs them across however many OS threads the VM has (typically 1× the CPU count).

B.2 Message passing

Processes communicate by sending messages:

```
Pid ! {hello, "world"}.
```

The receiving process gets the message in its mailbox and processes it via `receive`:

```
receive
  {hello, Name} -> io:format("hi ~p~n", [Name])
end.
```

Messages are asynchronous — the sender doesn't wait. Mailboxes are unbounded by default. `receive` can pattern-match selectively (skip messages that don't match the current expected shape).

B.3 gen_server

A standard pattern for "process that holds state and responds to queries." You implement the callbacks:

```
init(Args) ->
  {ok, InitialState}.

handle_call(Request, _From, State) ->
  {reply, Response, NewState}.
```

```
handle_cast(Notification, State) ->
  {noreply, NewState}.
```

Clients use `gen_server:call(Pid, Request)` (synchronous, blocks until the server replies) or `gen_server:cast(Pid, Notification)` (fire-and-forget).

For live coding: each Track is naturally a `gen_server`. "Set this track's pattern" is a `cast`. "What's currently playing" is a `call`.

B.4 Supervisors

A supervisor is a process whose only job is to start and restart child processes according to a strategy. If a child crashes, the supervisor restarts it. The user (or higher-level supervisor) doesn't have to handle the failure.

Strategies:

- `one_for_one`: only the crashed child restarts.
- `one_for_all`: all siblings restart together.
- `rest_for_one`: the crashed child and all started after it restart.

For live coding: a Track supervisor watches all tracks and restarts crashed ones with `one_for_one`. The user doesn't notice the failure; the broken pattern's effect was just one cycle of silence on that track.

B.5 Hot code loading

Erlang ships with the ability to swap a module's code in a running VM:

```
code:load_file(my_module).
```

Reads the latest `my_module.beam` file and installs it. Existing call frames in running processes finish on old code; new calls go to new code.

For live coding: the cell-fire loop is `compile cell` → `load_file` → `send pattern message to track`. The track was running the whole time. No restart, no state loss. This is essentially what GHCI's in-process eval gives you, but at the BEAM level rather than the language level.

B.6 Distributed Erlang

Two BEAM nodes with the right configuration can transparently exchange messages between processes on different machines:

```
Pid = spawn(somenode@otherhost, fun() -> do_something() end),
Pid ! {hello, "world"}.
```

The `!` operator works the same locally and remotely. This is what enabled Erlang's original telephony use cases (distributed switching) and what Riak (the distributed database) is built on.

For our use: probably overkill, but the property comes for free. Multi-machine collaborative live-coding setups don't require any new transport infrastructure — just two BEAM nodes gossip-clustered.